

ENTREGA FINAL RRNN



UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE CIENCIAS MATEMÁTICAS
MÁSTER EN INGENIERÍA MATEMÁTICA

ASIGNATURA: Redes Neuronales y Computación Paralela

ALUMNA: Siria Catherine Íñiguez Brito

PROFESOR: Valeri A. Makarov

CURSO: 2025-2026

Madrid, Junio de 2026

Contents

1	PROBLEMA I: Clasificación de imágenes MNIST con un perceptrón	2
1.1	Cálculo del Gradiente	2
1.2	Resultados Experimentales	3
1.2.1	Pareja 0 vs 4	3
1.2.2	Pareja 1 vs 7	4
1.2.3	Comparativa y Observaciones	5
1.3	Evolución del error de clasificación	5
2	PROBLEMA II: Clasificación de dígitos manuscritos caseros	7
2.1	Arquitectura y Entrenamiento de los Modelos	7
2.2	Pruebas con Dígitos Caseros	8
2.3	Análisis de Resultados y Comparativa	8
3	PROBLEMA III: Clasificación de reseñas de películas	9
3.1	Preprocesamiento y Representación de Datos	9
3.2	Arquitectura de la Red Neuronal	9
3.3	Configuración del Entrenamiento	10
3.4	Evaluación y Análisis de Resultados	10
3.4.1	Análisis de Errores	11
3.4.2	Análisis Cualitativo: Ejemplos de Clasificación	12

1 PROBLEMA I: Clasificación de imágenes MNIST con un perceptrón

El modelo consiste en un perceptrón que procesa una entrada $x \in \mathbb{R}^{784}$ con las siguientes etapas:

$$\begin{aligned}u &= \langle w, x \rangle + \theta \\y &= \sigma(u) \\z &= H(y - 0.5)\end{aligned}$$

Donde las definiciones principales son:

- **Función de coste:** $\mathcal{L}(w, \theta) = \frac{1}{N} \sum_{i=1}^N (c_i - y_i)^2$
- **Error de clasificación:** $E = \frac{1}{N} \sum_{i=1}^N |c_i - z_i|$
- **No linealidad:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Algoritmo de descenso del gradiente con momentos:**

$$\begin{cases} s_k = \beta s_{k-1} + (1 - \beta) \nabla \mathcal{L}(p_k) \\ p_{k+1} = p_k - \alpha s_k \end{cases}$$

El código para la resolución de este problema y sus diferentes experimentos se encuentra en el archivo `PracticaFinal_Ejercicio1_RRNN.m`.

1.1 Cálculo del Gradiente

Para que el algoritmo de optimización pueda actualizar los parámetros en cada iteración k , necesitamos calcular el gradiente $\nabla \mathcal{L}(p_k)$.

Para una muestra individual i , el flujo de derivación es el siguiente:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \underbrace{\frac{\partial \mathcal{L}}{\partial y_i}}_{1. \text{ Error}} \cdot \underbrace{\frac{\partial y_i}{\partial u_i}}_{2. \text{ Activación}} \cdot \underbrace{\frac{\partial u_i}{\partial w_j}}_{3. \text{ Entrada}}$$

1. **Derivada del coste (Error en salida):** Mide la diferencia entre la predicción y el objetivo. Es la señal que inicia el proceso de aprendizaje.

$$\frac{\partial \mathcal{L}}{\partial y_i} = \frac{\partial}{\partial y_i} (c_i - y_i)^2 = 2(y_i - c_i)$$

2. **Derivada de la activación:** Para la función sigmoide, su derivada respecto a la entrada neta u_i es:

$$\frac{\partial y_i}{\partial u_i} = \sigma'(u_i) = \frac{e^{-u_i}}{(1 + e^{-u_i})^2}$$

Si recordamos, la salida de la neurona se definió como $y_i = \sigma(u_i)$, así que podemos reescribir la derivada en términos de la propia salida:

$$\sigma'(u_i) = \sigma(u_i) (1 - \sigma(u_i)) = y_i(1 - y_i)$$

3. Derivada de la entrada neta:

- Para los pesos: $\frac{\partial u_i}{\partial w_j} = x_{ij}$. Esto indica que el ajuste de un peso depende del valor del píxel de entrada.
- Para el sesgo: $\frac{\partial u_i}{\partial \theta} = 1$. El sesgo afecta a la neurona por igual, independientemente de la imagen de entrada.

Finalmente, al promediar estas derivadas sobre el total de N muestras, obtenemos las componentes del vector gradiente $\nabla \mathcal{L}(p_k)$ que utiliza el algoritmo de momentos para actualizar el estado del modelo:

$$\nabla_w \mathcal{L} = \frac{2}{N} \sum_{i=1}^N (y_i - c_i) y_i (1 - y_i) \mathbf{x}_i^\top$$

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{2}{N} \sum_{i=1}^N (y_i - c_i) y_i (1 - y_i)$$

1.2 Resultados Experimentales

Para evaluar la robustez y capacidad de generalización del perceptrón, se ha realizado un estudio comparativo utilizando dos parejas de dígitos con distintas características: el par **0 vs 4** (distinción estructural clara) y el par **1 vs 7** (mayor similitud visual). Para asegurar la significancia estadística y minimizar el impacto de la inicialización aleatoria de los pesos, cada configuración ha sido ejecutada en 10 pruebas independientes, calculando la media y la desviación típica de la precisión obtenida. La metodología sigue una estrategia de bloques secuenciales manteniendo los parámetros restantes constantes: primero, se analiza la sensibilidad de la tasa de aprendizaje (α) con $\beta = 0.8$ y $E = 500$; segundo, el factor de momento (β) con $\alpha = 0.25$ y $E = 500$; y tercero, el número de épocas (E) con $\alpha = 0.25$ y $\beta = 0.85$. Finalmente, se presenta una matriz de resultados combinados para $E = 1000$.

1.2.1 Pareja 0 vs 4

Intuitivamente, esta pareja debería ser más fácil de clasificar porque las formas del ‘0’ y el ‘4’ son muy distintas entre sí.

Análisis de sensibilidad de α ($\beta = 0.8, E = 500$)

α	0.01	0.1	0.25	0.5
Media (%)	55.30	81.82	86.74	87.60
Desv. Típica (%)	10.40	2.10	0.91	0.35

Análisis de sensibilidad de β ($\alpha = 0.25, E = 500$)

β	0.50	0.80	0.85	0.90
Media (%)	86.67	86.78	86.64	86.10
Desv. Típica (%)	0.79	0.95	0.76	0.83

Impacto del número de épocas ($\alpha = 0.25, \beta = 0.85$)

Épocas	200	500	700	1000
Media (%)	81.36	86.87	87.15	87.46
Desv. Típica (%)	3.37	1.42	0.61	0.45

Matriz de resultados combinados: α vs β ($E = 1000$)

$\alpha \setminus \beta$	0.5		0.8		0.85		0.9	
	Med	Std	Med	Std	Med	Std	Med	Std
0.01	60.12%	6.88%	63.84%	7.15%	61.10%	8.07%	59.01%	11.88%
0.1	86.49%	1.47%	86.50%	1.05%	86.32%	1.42%	85.36%	0.91%
0.25	87.68%	0.44%	87.53%	0.64%	87.57%	0.55%	87.81%	0.61%
0.5	87.69%	0.23%	87.82%	0.24%	87.81%	0.09%	87.85%	0.27%

1.2.2 Pareja 1 vs 7

Por el contrario, esta pareja debería de ser más difícil porque el ‘1’ y el ‘7’ se parecen mucho en su forma y trazo. Como se ven casi iguales en muchas partes, al modelo seguramente le cuesta mucho más trabajo distinguirlos.

Análisis de sensibilidad de α ($\beta = 0.8, E = 500$)

α	0.01	0.1	0.25	0.5
Media (%)	56.28	75.96	81.76	81.77
Desv. Típica (%)	9.56	3.45	1.23	0.96

Análisis de sensibilidad de β ($\alpha = 0.25, E = 500$)

β	0.50	0.80	0.85	0.90
Media (%)	80.83	80.64	80.68	80.35
Desv. Típica (%)	1.91	1.39	1.13	1.43

Impacto del número de épocas ($\alpha = 0.25, \beta = 0.85$)

Épocas	200	500	700	1000
Media (%)	76.36	81.32	81.53	81.32
Desv. Típica (%)	2.45	1.44	1.19	0.68

Matriz de resultados combinados: α vs β ($E = 1000$)

	0.5		0.8		0.85		0.9	
$\alpha \setminus \beta$	Med	Std	Med	Std	Med	Std	Med	Std
0.01	52.75%	6.91%	55.51%	10.08%	54.08%	7.11%	55.78%	5.61%
0.1	80.08%	0.67%	80.29%	1.78%	80.51%	1.73%	80.15%	2.03%
0.25	81.27%	0.95%	81.16%	0.61%	81.60%	1.07%	81.54%	0.83%
0.5	81.65%	0.61%	81.22%	0.43%	81.67%	0.48%	81.67%	0.48%

1.2.3 Comparativa y Observaciones

La diferencia en la precisión máxima alcanzada (aproximadamente un 87% para el par 0-4 frente a un 81% para el par 1-7) evidencia la principal limitación del perceptrón: al ser un modelo lineal, su capacidad para separar clases está restringida a trazar un único hiperplano en el espacio de datos. Mientras que los parámetros α y β logran optimizar el ajuste del modelo, el “techo” de rendimiento viene impuesto por la propia naturaleza de los datos. En el caso del par 0-4, la marcada diferencia estructural entre los dígitos permite una separación eficaz. Por el contrario, el par 1-7 presenta una alta similitud visual y una mayor superposición de características, lo que impide que el perceptrón logre distinguirlos con éxito.

1.3 Evolución del error de clasificación

Para analizar el comportamiento del perceptrón, evaluamos inicialmente el entrenamiento con $E = 500$ épocas ($\alpha = 0.25, \beta = 0.85$). Posteriormente, extendimos el entrenamiento a $E = 1000$ para observar si el modelo lograba una convergencia más profunda o si, por el contrario, alcanzaba un límite.

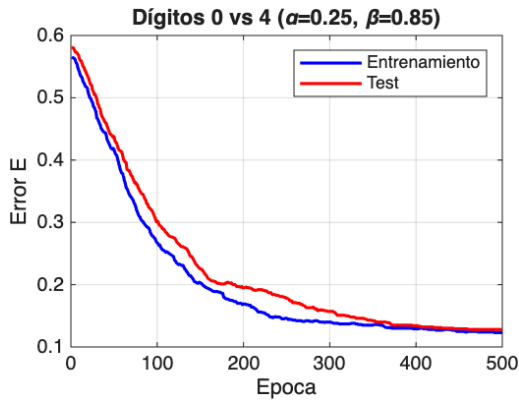


Figure 1: Pareja 0 vs 4: Evolución del error $E = 500$.

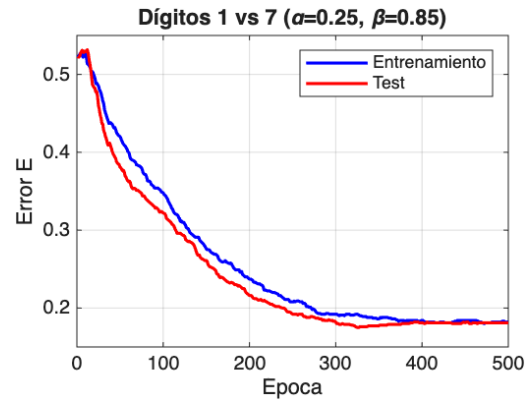


Figure 2: Pareja 1 vs 7: Evolución del error $E = 500$.

Tras completar las 500 épocas de entrenamiento, se evaluó la capacidad de generalización del modelo sobre los conjuntos de prueba. Los resultados finales de precisión obtenidos para cada par de dígitos son los siguientes:

Table 1: Precisión final del clasificador.

Pareja de Dígitos	Precisión (%)
0 vs 4	87.20%
1 vs 7	81.90%

Al incrementar el entrenamiento a 1000 épocas, observamos cómo se estabiliza la curva de error:

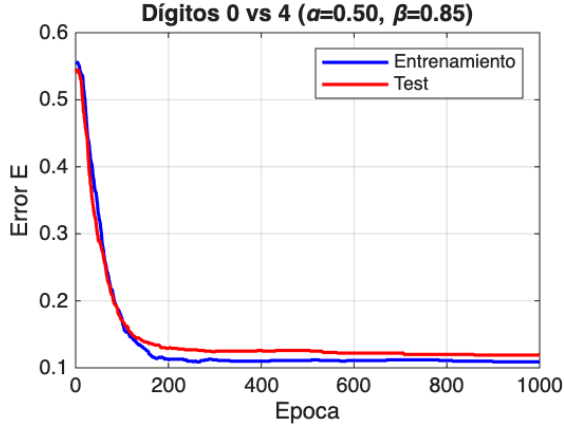


Figure 3: Pareja 0 vs 4: Evolución del error $E = 1000$.

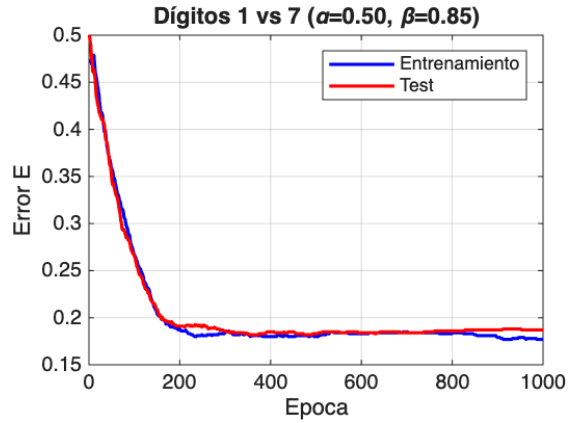


Figure 4: Pareja 1 vs 7: Evolución del error $E = 1000$.

- **Dinámica de convergencia:** En ambos casos, el error decae rápidamente durante las primeras 300 épocas, estabilizándose de forma notable al alcanzar las 1000. Esto valida la eficacia del descenso del gradiente con momentos para encontrar un mínimo estable en el espacio de parámetros.
- **Generalización y robustez:** La distancia observada entre la curva de entrenamiento (azul) y la de test (roja) indica que el modelo generaliza bien, sin presentar signos de sobreajuste.
- **Limitaciones por similitud morfológica:** La brecha en la precisión final (88.00% para 0-4 frente a 81.30% para 1-7) confirma que la similitud visual entre el 1 y el 7 dificulta la separación lineal. El modelo llega a un límite donde el error residual se mantiene constante, demostrando que la arquitectura del perceptrón es insuficiente para resolver ambigüedades visuales complejas que requieren fronteras de decisión no lineales.

2 PROBLEMA II: Clasificación de dígitos manuscritos caseros

En este ejercicio se evaluará la capacidad de generalización de arquitecturas de redes neuronales avanzadas mediante la construcción y entrenamiento de dos modelos distintos: una red neuronal densamente conectada (*fully connected*) y una red neuronal convolucional (CNN), utilizando como base de referencia el conjunto de datos MNIST, para posteriormente poner a prueba su robustez mediante la clasificación de un conjunto de cifras manuscritas “caseras” escaneadas, finalizando con un análisis descriptivo y una comparativa de los resultados obtenidos con cada arquitectura.

2.1 Arquitectura y Entrenamiento de los Modelos

La resolución del ejercicio se encuentra en el archivo `PracticaFinal_Ejercicio2_RRNN.m`. Para este problema, se diseñaron dos arquitecturas distintas para la clasificación de imágenes de 28×28 píxeles, utilizando como base el conjunto de datos MNIST.

Para facilitar que los modelos sean capaces de reconocer nuestros dígitos “caseros” —los cuales presentan variabilidad intrínseca en inclinación, grosor de trazo, desplazamiento espacial y escala—, se ha integrado un *Data Augmentation* mediante el objeto `imageDataAugmenter`. Esta técnica expande artificialmente el conjunto de entrenamiento aplicando rotaciones (-15° a 15°), traslaciones (-3 a 3 píxeles) y ajustes de escala (0.9 a 1.1).

A. Red Fully Connected (Densa)

Capa	Configuración	Dropout
Input	28×28	-
FC 1	1024 neuronas + BN + ReLU	0.4
FC 2	512 neuronas + BN + ReLU	0.4
FC 3	256 neuronas + ReLU	-
Output	10 neuronas + Softmax	-

Entrenamiento: Se optó por el optimizador **Adam** (30 épocas), para poder ajustar dinámicamente la tasa de aprendizaje, complementado con una regularización L2 (0.001) para penalizar pesos excesivos y estabilizar la convergencia.

B. Red Convolucional (CNN)

Bloque	Operaciones	Output
Bloque 1	Conv (3×3 , 32 filtros) + BN + ReLU + MaxPool	14×14
Bloque 2	Conv (3×3 , 64 filtros) + BN + ReLU + MaxPool	7×7
Bloque 3	Conv (3×3 , 128 filtros) + BN + ReLU	7×7
Clasificador	FC 256 + ReLU + Dropout (0.3)	256
Salida	FC 10 + Softmax + Clasificación	10

Entrenamiento: Se utilizó **SGDM** con momento (0.9), que favorece la acumulación de velocidad en la dirección correcta del gradiente, permitiendo un aprendizaje más eficiente durante 20 épocas con una tasa de aprendizaje constante ($\alpha = 0.01$).

2.2 Pruebas con Dígitos Caseros

Para evaluar la generalización en entornos reales, se ha utilizado una carpeta denominada *MisDigitos*, que contiene imágenes individuales de dígitos manuscritos. Cada imagen ha sido procesada mediante un algoritmo de segmentación y centrado: primero se binariza e invierte la imagen para obtener un trazo blanco sobre fondo negro, posteriormente se calcula el área ocupada por el dígito mediante *find*, y finalmente se recorta y redimensiona a 28×28 añadiendo un margen de píxeles para asegurar el formato requerido por las redes.

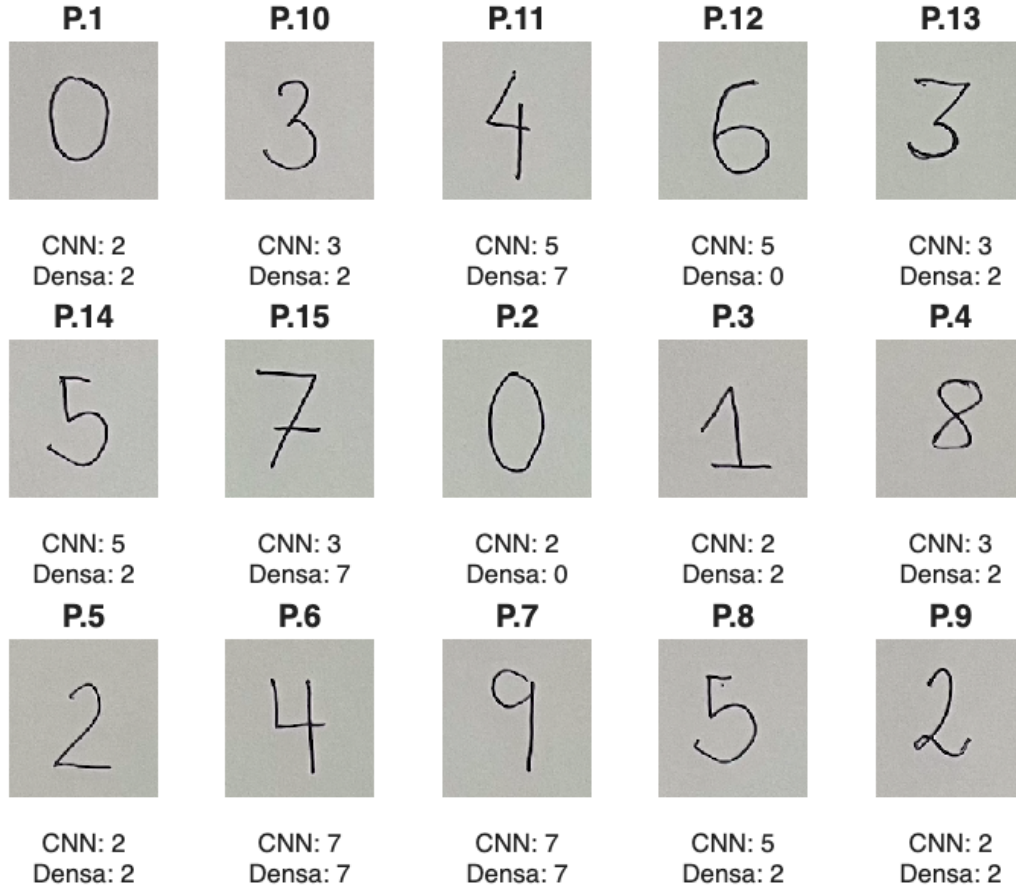


Figure 5: Clasificación de dígitos caseros.

2.3 Análisis de Resultados y Comparativa

Tras evaluar la red densa y la CNN, se observa que ambas alcanzan un rendimiento muy bueno dentro del conjunto de MNIST, logrando precisiones en el conjunto de test del **98.08%** y **99.86%** respectivamente; sin embargo, este éxito no se traslada al clasificar los dígitos caseros, pues la precisión disminuye hasta alcanzar **4/15 aciertos** para la red densa y **6/15** para la CNN, lo que puede sugerir que el modelo convolucional posee una robustez superior al extraer características jerárquicas espaciales frente al análisis píxel a píxel de la red *fully connected*. Esta brecha sugiere que a pesar del uso de *Data Augmentation*, las redes presentan cierta dependencia a la distribución específica de MNIST, mostrando sensibilidad ante variaciones en el grosor del trazo, el contraste y el centrado manual de las capturas caseras.

3 PROBLEMA III: Clasificación de reseñas de películas

El objetivo de este ejercicio es aplicar técnicas de aprendizaje profundo al análisis de sentimientos mediante el procesamiento de lenguaje natural (NLP). Para ello, se entrena y evalúa un modelo de clasificación utilizando la base de datos `BBDD_resenas.mat` y el uso de modelos de representación vectorial del lenguaje (*word embeddings*) preentrenados (`wiki-news-300d-1M.vec`). El flujo de trabajo abarca desde el preprocesamiento y vectorización de los textos hasta el diseño, entrenamiento y evaluación de una red neuronal, finalizando con un análisis de los resultados obtenidos.

Para la resolución de este ejercicio, se implementó el código que se encuentra en el fichero `PracticaFinal_Ejercicio3_RRNN.m`.

3.1 Preprocesamiento y Representación de Datos

Para el desarrollo del modelo se ha utilizado una base de datos compuesta por un total de 25.000 reseñas con 12.500 muestras positivas y 12.500 muestras negativas. Este balanceo es clave para asegurar que el clasificador no desarrolle sesgos hacia una polaridad específica.

El preprocesamiento es crítico en NLP para reducir el ruido. Se han seguido los siguientes pasos:

- **Recorte de texto:** Se limitaron las reseñas originales a los primeros 500 caracteres para optimizar la carga computacional sin perder la esencia del sentimiento.
- **Normalización:** Conversión a minúsculas, eliminación de puntuación y palabras de longitud menor a 2 caracteres.
- **Filtrado:** Eliminación de *stop words* y términos específicos del tema (“movie”, “film”, “br”).
- **Lematización:** Reducción de las palabras a su raíz o lema mediante `normalizeWords`.
- **Vectorización:** Se utilizó el modelo preentrenado `wiki-news-300d-1M` de 300 dimensiones. Los textos se convirtieron en secuencias de vectores con un `maxTokens` de 100.

3.2 Arquitectura de la Red Neuronal

Se diseñó una arquitectura basada en Redes Neuronales Convolucionales (CNN) 1D, para capturar patrones en secuencias de texto:

- **Capa de Entrada:** Recibe secuencias de dimensión 300.
- **Bloques Convolucionales:** Dos capas `convolution1dLayer` (con 128 y 64 filtros respectivamente) seguidas de `Batch Normalization` y `ReLU` para acelerar la convergencia y mejorar la estabilidad.

- **Agregación:** Una capa `globalMaxPooling1dLayer` que extrae las características más relevantes de toda la secuencia.
- **Regularización y Salida:** Una capa `Dropout` al 40% para evitar el sobreajuste, conectada a una capa `Fully Connected` de 2 neuronas con activación `Softmax` para la clasificación binaria.

3.3 Configuración del Entrenamiento

El entrenamiento se configuró de la siguiente manera:

- **Optimizador:** Adam con una tasa de aprendizaje inicial de 0.0005.
- **Lote (Mini-batch):** 32 muestras.
- **Épocas:** 10 iteraciones completas sobre el conjunto de datos.
- **Validación:** Se reservó un 10% del conjunto de entrenamiento para monitorizar el rendimiento.

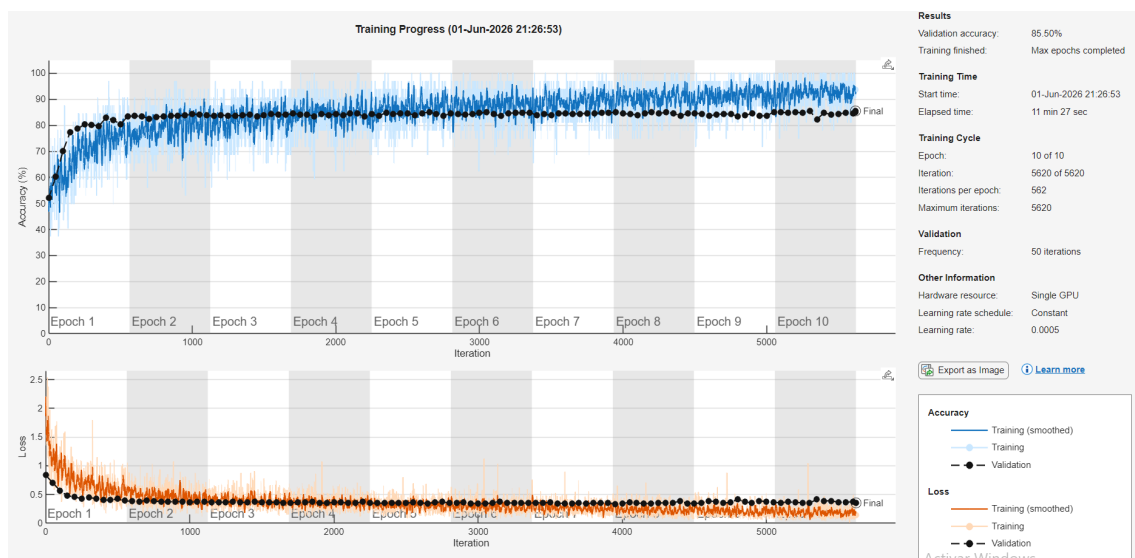


Figure 6: Progreso del entrenamiento: Evolución de la precisión (Accuracy) y la pérdida (Loss).

3.4 Evaluación y Análisis de Resultados

El rendimiento del modelo se presenta en comparación con los resultados obtenidos por el modelo base sugerido en los apuntes (ver Tabla 2).

Como se puede observar, la arquitectura implementada ha logrado mejorar la precisión en ambos conjuntos, especialmente en el de prueba. A pesar de la mejora de casi 6 puntos porcentuales en el conjunto de test, se observa una diferencia entre el acierto en *train* y *test*. Esto podría indicar un fenómeno de sobreajuste (*overfitting*), aunque se haya intentado mitigar parcialmente mediante la inclusión de capas de `Dropout` y `Batch Normalization`.

Conjunto de Datos	Precisión Modelo Base	Precisión Modelo Nuevo
Entrenamiento (Train)	96.6%	98.16%
Prueba (Test)	78.9%	84.70%

Table 2: Comparativa de resultados: Modelo apuntes vs. Modelo final.

3.4.1 Análisis de Errores

Analizando la distribución de errores de la Figura 7, se realiza una comparativa entre los tres histogramas:

- **Distribución General (Azul):** Muestra el conjunto de test completo, donde se aprecia una concentración de reseñas en los extremos para las puntuaciones de 1 y 10.
- **Muestras Mal Clasificadas (Rojo):** Este gráfico revela que el modelo presenta su mayor tasa de fallo en las puntuaciones intermedias (especialmente en los valores 4 y 7). Las reseñas con sentimientos “mixtos” o moderados son más difíciles de categorizar binariamente, ya que el lenguaje utilizado suele ser más ambiguo.
- **Muestras Bien Clasificadas (Verde):** Se observa una alta densidad en las puntuaciones extremas (1 y 10). Esto demuestra que, cuando el usuario utiliza un léxico claramente polarizado y contundente, la red neuronal alcanza una precisión muy superior.

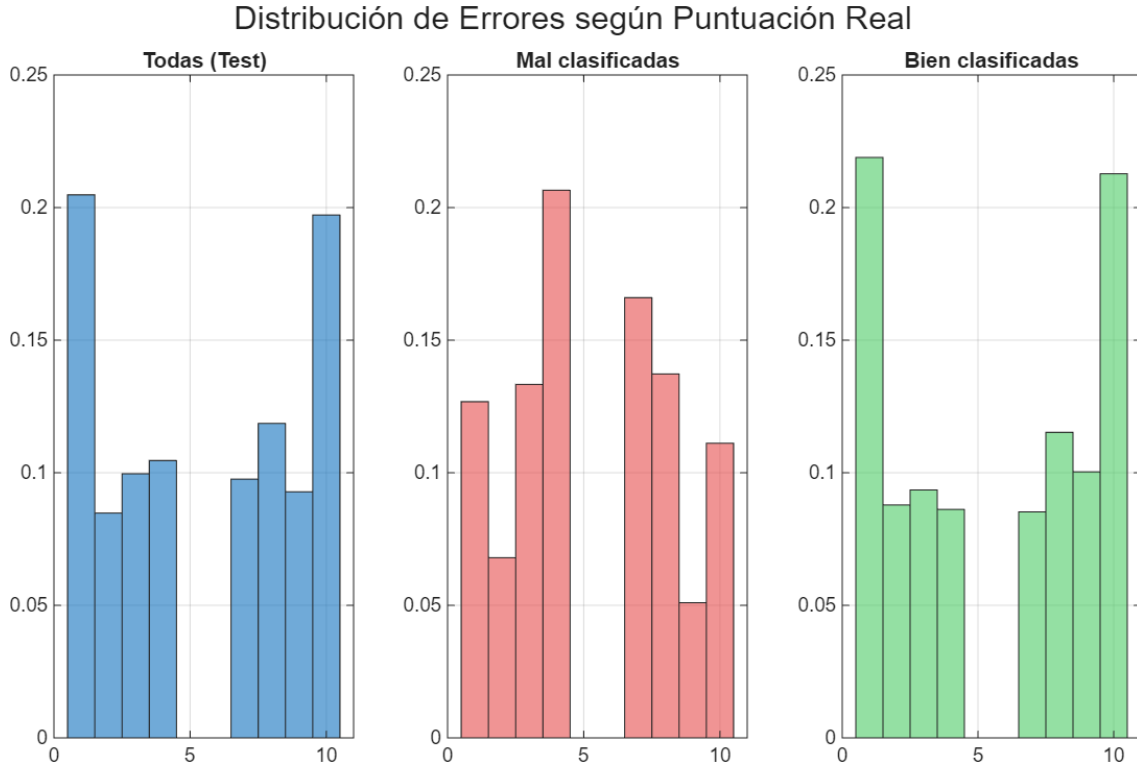


Figure 7: Distribución de la puntuación real: total de muestras en test (azul), errores del modelo (rojo) y aciertos (verde).

3.4.2 Análisis Cualitativo: Ejemplos de Clasificación

Para comprender mejor el comportamiento del modelo, se han extraído dos ejemplos del conjunto de test:

- **Ejemplo Mal Clasificado (Falso Positivo):**

“...Garp is born to a formidable unmarried mother... Williams gives a cherub-faced performance... This movie was able to attract some reasonable attention in 1982...”

Score real: 4 — **Clase real:** Negativa (1) — **Predicción:** Positiva (2).

En este caso, la reseña tiene una puntuación de 4, pero el texto utiliza adjetivos que el modelo puede interpretar como positivos en un contexto aislado, tales como *“formidable”*, *“reasonable attention”* o *“cherub-faced performance”*. El modelo falla al no captar el tono crítico general o el sentido de frases como *“his talent was wasted”*, probablemente debido al límite de 100 tokens o a la ambigüedad de las palabras descriptivas de la trama.

- **Ejemplo Bien Clasificado (Verdadero Positivo):**

“...it’s actually a very remarkable film... hand painted... The dance itself is very hypnotic and much like a piece of amazing performance art...”

Score real: 8 — **Clase real:** Positiva (2) — **Predicción:** Positiva (2).

Aquí, el lenguaje es explícitamente positivo y entusiasta. Palabras como *“remarkable”*, *“amazing”*, *“hypnotic”* y *“visually compelling”* tienen una fuerte carga semántica en el modelo de *embedding*, lo que permite a la red identificar patrones claros de satisfacción del usuario.